

CIT316 – Advanced Software Design

Tutorial – 07

Part A

a) Define Software Quality Assurance (SQA)

SQA means ensuring that when software is developed, it follows the correct requirements, standards, and processes. Its main goal is to prevent defects and improve the quality of the final product.

b) Identify and explain three activities included in SQA.

- **Process Definition and Improvement**
SQA involves defining standard development and continuously improving them. This ensures consistency and reduces errors across projects.
- **Software Reviews and Audits**
Regular reviews and audits are conducted to check whether the software and processes comply with defined standards. These help identify issues early.
- **Testing and Validation Oversight**
SQA ensures that proper testing strategies (unit testing, integration testing, system testing) are planned and executed effectively to verify that the software meets requirements.

c) Explain the difference between Defect prevention and Defect detection.

Defect prevention	Defect Detection
Avoid defects before they occur	Find defects after they occur
Improve processes, training, standards	Testing, debugging, inspections
Early (during design & development)	Later (during testing or after release)
Using coding standards, developer training	Running test cases, bug tracking

d) Describe why SQA is important in modern software systems.

- **Financial Savings:** It is much cheaper to fix a bug during the design phase than after the software has launched.

- **Security & Compliance:** It ensures data is protected and the system meets legal standards (like GDPR), preventing lawsuits and hacks.
- **Managing Complexity:** In systems using micro services and APIs, SQA ensures that changing one part doesn't break the entire "web" of services.
- **User Loyalty:** High-quality, fast, and bug-free software keeps users from switching to a competitor.
- **Cost Efficiency:** Fixing defects early is far cheaper than fixing them after deployment.
- **User Expectations:** Users expect fast, secure, and bug-free applications.

Part B

a) Identify and explain the four levels of testing:

- **Unit Testing** - Testing individual components or functions in isolation to ensure they work correctly.
- **Integration Testing:** Testing combined units to verify that they interact and communicate properly.
- **System Testing:** Testing the complete, integrated system to check if it meets all specified requirements.
- **Acceptance Testing:** Testing conducted by users or clients to confirm the system meets their needs and is ready for release.

b) For each testing level, provide one practical example related to an online shopping system.

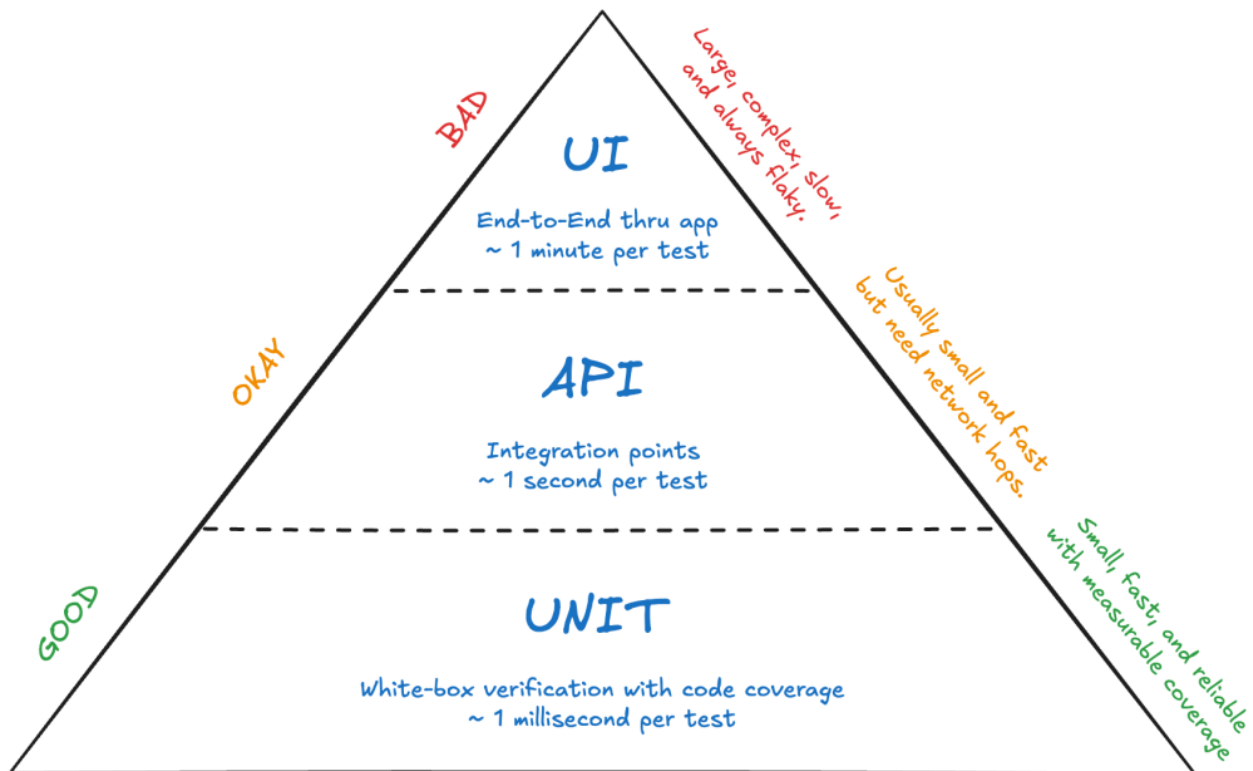
- **Unit Testing:**
Testing a function that calculates the total price of items in the cart (including discounts and taxes) to ensure it returns the correct amount.
- **Integration Testing:**
Testing whether the shopping cart correctly communicates with the payment gateway to process payments successfully.
- **System Testing:**
Testing the entire workflow—user logs in, searches for a product, adds it to the cart, checks out, and receives an order confirmation.
- **Acceptance Testing:**
A client or end user verifies that they can place an order, choose delivery options, and complete payment according to their requirements before approving the system.

c) Compare Unit Testing vs Integration Testing based on Scope, Purpose, and Complexity

	Unit Testing	Integration Testing
Scope	Focuses on a single component in isolation.	Focuses on multiple components working together as a group.
Purpose	Ensures that each individual unit of code works correctly.	Ensures that different modules interact and communicate correctly.
Complexity	Simple and easy to execute since it deals with small, isolated pieces of code.	More complex because it involves multiple components and their interactions.
Summary	Small scope, checks individual parts, low complexity.	Broader scope, checks interactions, higher complexity.

Part C

a) Draw and label the Testing Pyramid.



b) Explain why Unit tests are many and fast, and UI tests are few and slow.

Unit tests are the high speed backbone of a testing suite. By isolating individual functions or methods and removing dependencies on databases or networks, they execute almost instantaneously. Their simplicity allows developers to write hundreds of tests quickly, ensuring deep code coverage with minimal overhead.

UI tests prioritize the end-to-end user experience, verifying how the entire system from the interface to the backend functions together. Because they rely on live environments like browsers and external networks, they are significantly slower and more fragile than unit tests. Due to this complexity and the high cost of maintenance, they are typically reserved for a few critical "happy path" scenarios.

c) What problems occur if a system has too many UI tests or too few unit tests?

If a system has too many UI tests, development becomes slow because these tests take a long time to run and are harder to maintain. Frequent UI changes can easily break tests, causing instability and wasting time fixing them. On the other hand, having too few unit tests is risky because small parts of the code are not properly tested. This allows bugs to go unnoticed until later stages, making them harder and more expensive to fix.

Part D

a) Explain the CI process flow after code is pushed to GitHub.

After code is pushed to GitHub, the Continuous Integration (CI) process is automatically triggered (usually by tools like GitHub Actions or Jenkins). First, the CI system checks out the latest code from the repository. Then it installs dependencies and builds the project. After that, automated tests (unit tests and sometimes integration tests) are executed to verify the code. If all tests pass, the build is marked successful and may be deployed or packaged. If any test fails, the CI process stops and reports errors back to the developer.

b) List and explain Build, Test, and Deploy stages.

Build Stage:

In this stage, the source code is compiled and converted into an executable application. The system also installs all required dependencies and checks for basic errors. The goal is to ensure that the code can successfully build without issues.

Test Stage:

After building, automated tests such as unit tests and integration tests are executed. This stage verifies that the application works correctly and that new changes have not broken existing functionality. If any test fails, the process stops and reports errors.

Deploy Stage:

If the build and tests are successful, the application is deployed to a target environment such as a staging or production server. This stage makes the application available for users or further testing, often using automated deployment tools.

c) Identify two benefits of CI in team development

One major benefit of Continuous Integration (CI) in team development is early bug detection. Since code is tested frequently after each change, issues are found quickly before they become larger problems.

Another benefit is better collaboration among team members. CI ensures that all developers integrate their code regularly, reducing merge conflicts and keeping the project stable and consistent.

d) Explain how CI helps in early bug detection and faster delivery.

Continuous Integration (CI) helps in early bug detection by automatically running tests every time code is pushed to the repository. This means errors are identified immediately after changes are made, rather than being discovered later in development or production. CI also supports faster delivery because the code is always in a tested and working state, allowing teams to release updates more frequently and with less manual effort in testing and integration.

- e) Design a simple CI pipeline diagram including Developer, Git repository, Automated testing, and Deployment.

